

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 5 : Session – 5 : SLO – 1 | Graph Implementation Using Array — Insertion of Node and Edge

■ I. Fill in the Blanks

- Inserting a node in an adjacency matrix requires adding a new ROW AND COLUMN to the matrix.
- Inserting a node in an adjacency list requires adding a new LIST (head node).
- Inserting an edge in an adjacency matrix involves setting 1 in the corresponding cell.
- In an undirected graph, inserting an edge between A and B requires updating both $MATRIX[A][B]$ and $MATRIX[B][A]$.
- In an adjacency list, an edge insertion means adding the destination vertex to the LIST of the source vertex.

■ II. True or False

- Inserting a new node increases the size of an adjacency matrix — TRUE.
- Edge insertion in an adjacency list is slower than in an adjacency matrix — FALSE (matrix is $O(1)$, list is $O(1)$ too).
- In an undirected graph, each edge is stored twice in an adjacency list — TRUE.
- You can insert multiple edges between two nodes in a simple graph — FALSE.

■ III. Pseudocode — Insertion

- Insertion of a Node (Adjacency List):
 1. Create a new VERTEX (node) for the new vertex.
 2. Initialize its adjacency list to NULL (empty).
 3. Update the total number of vertices.
- Insertion of an Edge (Adjacency List):
 1. Check if both vertices exist.
 2. Add the destination vertex to the adjacency list of the SOURCE vertex.
 3. If the graph is undirected, also add the source vertex to the adjacency list of the DESTINATION vertex.

■ IV. Spot the Error

- Mistake: $matrix[v][v] = 1$ sets a self-loop instead of the reverse edge.
- Correct it to: $matrix[u][v] = 1$; $matrix[v][u] = 1$;

■ V. Complete the C Code

- Adjacency Matrix:

```
void addEdge(int graph[10][10], int u, int v) {  
    graph[u][v] = 1;  
    graph[v][u] = 1; // undirected graph  
}
```
- Adjacency List:

- void addEdge(struct Node* adjList[], int u, int v) {
- struct Node* newNode = createNode(v);
- newNode->next = adjList[u];
- adjList[u] = newNode;
- // For undirected graph:
- struct Node* newNode2 = createNode(u);
- newNode2->next = adjList[v];
- adjList[v] = newNode2;
- }

■ VI. Predict the Output

- After addEdge(graph, 0, 1) and addEdge(graph, 0, 2):
- 0 1 2
- 0 [0 1 1]
- 1 [1 0 0]
- 2 [1 0 0]